
What Counts as a Mistake? Annotating Recitation Events in Quran Memorization Transcripts

Anonymous Author(s)

Affiliation

Address

email

Abstract

1 Checking Quran recitation from an ASR transcript requires distinguishing unre-
2 solved mistakes from repetitions, repairs, opening formulas and accepted spelling
3 differences. We report a completed human annotation of 100 production recording
4 cases, covering 348 scored units and 162 localized events across ten combined
5 labels. An executable evaluator scores labels and word positions together. A plain
6 diff reaches label-aware F1 0.525 and localization F1 0.826; adapted production
7 cleaner/alignment components reach 0.518 and 0.786. Their exact-span F1 scores
8 are both 0.505. Correcting the adapter’s word coordinates recovers all five anno-
9 tated repetition events, showing why annotation interfaces must be checked before
10 interpreting baseline failures. In a preliminary pilot, seven single 20-minute-budget
11 runs across three coding agents and seven underlying models span label-aware
12 F1 0.143 to 0.892: six runs land far above every baseline, and one collapses
13 below the naive-diff baseline from a missing normalization step, an informative
14 failure as much as the successes are. The dataset and scorecard support the larger,
15 repeated-run agent comparison this pilot precedes.

16 1 The distinction the task turns on

17 Quran memorization is normally practised with a listener who corrects the reciter. An application
18 that stands in for that listener transcribes with commercial ASR and then compares the transcript
19 against the reference. Edit distance locates differences but does not determine their interpretation.
20 In our production data 74% of reviewed ayah units carry no event at all, and among those that do, a
21 substantial share are things a human listener would never mention: the reciter repeated a phrase after
22 a breath, said a word wrongly and immediately fixed it, or used a written form differing from the
23 Uthmani orthography without differing in what was recited. Flagging these as unresolved mistakes
24 can give the learner misleading feedback; we do not measure the relative user cost here.

25 We therefore annotate transcript-against-reference differences as typed, localized *events*, and we
26 judge the text rather than the speaker: a label records what differs between transcript and reference,
27 not whether the reciter or the ASR caused it. Unwritten vowel and tajweed errors lie outside a
28 transcript-only rubric and outside this work.

29 2 Taxonomy and annotation

30 **Combined labels.** An earlier pilot used two parallel fields, an event type and a be-
31 nign/mistake verdict, plus an uncertain escape hatch. Reviewing real recordings made
32 both untenable: the verdict was never independent of the type in practice, and uncertain

absorbed exactly the cases that needed a decision. The rubric now uses one combined label per event, `substitution_mistake`, `omission_mistake`, `insertion_mistake`, `substitution_corrected`, `omission_corrected`, `repetition_benign`, `letters_benign`, `spelling_benign`, `basmala_benign` and `isti3adha_benign`, with `clean` for a reviewed unit carrying no event. A unit reviewed and found clean is not the same as a unit not yet reviewed, and the data distinguishes them.

Four decisions that took adjudication. *Omissions* include missing beginnings, interiors and endings, with no exemption for the final ayah of a recording: stopping early still leaves reference material absent from the transcript, and a reviewer cannot infer intent from recording position. *Repeats* span the original phrase together with its extra occurrence against a single reference copy, aligning their localization with corrected events; a wrong-then-correct attempt is a corrected event, while a correct-then-incorrect restatement is a `substitution_mistake` spanning both attempts, with the order recorded in the note. *Spelling* differences removed by an agreed context-independent normalization receive no event, and only a residual form needing contextual acceptance receives `spelling_benign`; the accepted contexts are enumerated rather than generalized, so the category cannot quietly become blanket final-letter deletion. *Localization* uses verbatim words and half-open spans into the original whitespace token arrays, with an empty hypothesis span anchoring an omission and an empty reference span anchoring an insertion.

Scale and composition. All 100 selected recording cases are human-approved: 314 ayah units, 233 of them clean, 115 within-ayah events and 47 opening-formula annotations, over 274 ayahs in 38 surahs. The event distribution is uneven by nature, from 62 `substitution_mistake` down to one `insertion_mistake` and one `omission_corrected`, and any evaluation has to survive that. Ten learner identities contribute, one of them 46 recordings. Selection was purposive, so these counts describe the annotated set; they do not estimate error prevalence or imply generalization to unseen reciters. Three further approved recordings remain outside the scored set; their transcript-only copies join the development candidates, with annotations archived privately. We verified identifiers and transcripts against both source exports and excluded punctuation-normalized duplicates and redundant clean subpassages. Gold answers and selected membership remain in an owner-held private bundle; this draft does not release a gold sample.

3 An executable scorecard

Systems receive the reviewed ayah split, the ayah ids and the exact references, so detection and segmentation are supplied rather than scored; that task is treated in a companion submission. Per unit a system returns events of the form `{label, hyp_span, ref_span}`, with half-open spans into the transcript and reference token arrays.

Opening formulas are scored. Each record’s *isti’adhah* and *basmala* are presented as their own unit, with the formula as the transcript and an empty reference; a compound opening yields two events over one token array. Deciding that a *basmala* preceding the ayah is benign rather than an insertion is exactly the judgment under test, and excluding it would hide the failure mode that turns out to matter most.

Matching. Predictions and gold events are paired one-to-one within a unit by maximum total similarity, so a single broad prediction is credited with at most one gold event and the remainder count as misses. A pair is eligible only when *both* spans overlap, $\min(\text{sim}_{hyp}, \text{sim}_{ref}) \geq 0.50$; averaging the two, as an earlier version did, lets an exact reference span carry a match while the hypothesis span points somewhere else entirely. Events are validated for known labels, integer endpoints, token bounds and label-specific empty spans. Invalid predictions count as false positives. Empty spans match only empty anchors, exactly or within one token; they never match selected words. Pairing ignores labels so localization can be reported separately. Larger units with more than seven events on either side use deterministic greedy one-to-one matching.

Scores. The primary measure is label-aware event F1: a paired prediction is a true positive only when its label also matches, and a mislabeled pair counts once as a false positive and once as a false negative. We report micro F1 as the headline, exact-span F1 requiring identical endpoints, macro F1 across labels present in gold, and localization precision/recall/F1 with labels dropped. An application-facing cost is reported separately in raw event counts, (*r* missed + false flags) per 100 units, at both $r = 1$ and $r = 2$, because no exchange rate between a missed mistake and an

unnecessary flag is empirically established. Neither rate ranks systems and the ordering is unchanged from 1:1 to 5:1; the rate matters only in that predicting nothing wins below roughly 0.77:1. Predicting nothing gives F1 0 and the gold annotation gives 1. Evaluator v2.1 passes synthetic coordinate, label, anchor and adapter regression tests and private gold oracle checks. Sweeping the tolerance from 0.05 to 1.00 moves baseline micro F1 by about four points and leaves exact-span F1 unchanged, since diff-based baselines emit exact spans; 0.50 is chosen to be explainable rather than tuned. One-token anchor slack changes no baseline score across 0 to 10 and is retained because the index of an omission between two tokens is genuinely ambiguous. A system predicting looser spans will depend on the tolerance more than these baselines do.

4 What existing systems do

System	micro F1	exact F1	macro F1	loc F1	review cost
Predict nothing	0.000	0.000	0.000	0.000	51.2
Plain diff	0.525	0.505	0.178	0.826	23.3
Adapted production components	0.518	0.505	0.284	0.786	26.7
Codex, GPT-6-Astra (pilot)	0.892	0.862	0.701	0.934	8.1
Claude Code, Opus 5 (pilot)	0.880	0.849	0.660	0.931	9.2
Codex, GPT-5.6-Sol (pilot)	0.875	0.863	0.631	0.938	9.5
Claude Code, Sonnet 5 (pilot)	0.856	0.825	0.609	0.912	9.8
Antigravity, Gemini 3.6 Flash (pilot)	0.797	0.773	0.387	0.878	14.1
Antigravity, Gemini 3.1 Pro (pilot)	0.795	0.677	0.502	0.873	10.9
Antigravity, gpt-oss-120B (pilot)	0.143	0.121	0.017	0.187	169.3

Table 1: Evaluator v2.1: 348 units, 162 gold events. No baseline or pilot run crashes; the gpt-oss-120B pilot run produces 22 invalid events, discussed below. The seven pilot rows are one 20-minute-budget development run per agent/model pair, not the preregistered comparison of Section 5.

The plain diff locates 126 of 162 gold events (recall 0.778), with localization F1 0.826. Its label-aware recall is 0.494 and F1 is 0.525. These quantities differ: localization F1 is not the percentage of gold events found. It reaches per-label F1 0.94 for omissions and 0.85 for substitutions, and zero for the other eight labels. It emits 46 insertion predictions, 34 of them on opening-text units, whereas gold has one insertion event. Compound openings contain two gold formulas but a diff can return one insertion spanning both.

The second baseline adapts existing production cleaner and alignment components to the new event schema; the deployed application does not natively expose it. The adapter retains original token identities through deletion, uses alignment indices and reference offsets, and maps cleaner notes to reference phrases and both attempts. Its explicit heuristic interprets a stumble as a repair only when the following equally long surviving span matches the reference; unlocalizable removed fragments remain extra text. Production’s partial-start scoring policy is retained. These choices are part of the baseline, not additional gold decisions.

This adapter predicts five repetitions and recovers all five (F1 1.0); six corrected-substitution predictions recover one of four gold events (F1 0.20). It has nonzero F1 on four labels and zero on six, and never predicts either opening-formula label, letters, contextual spelling or omission corrections. The earlier adapter lost valid repetition credit through incorrect coordinates. The remaining label gaps motivate explicit event modeling, but these results do not establish that cleaning itself causes the errors or measure every feature of the deployed application. Each baseline retains its own comparison normalization, which is an algorithm choice rather than a change to the gold rubric.

A preliminary agent pilot. Each of three coding agents (Codex, Antigravity, Claude Code) was given the unlabelled 127-recording training release, the annotation guide and 20 constructed teaching examples with answers, a 20-minute development budget, and no access to gold or the internet; each froze one `solution.py` that we then ran, unmodified, over the private gold set. Codex ran once each under GPT-5.6-Sol and GPT-6-Astra; Antigravity ran once each under Gemini 3.1 Pro, Gemini 3.6 Flash and gpt-oss-120B; Claude Code ran once each under Opus 5 and Sonnet 5. Seven runs, seven models, one run per model: this is not the repeated-run comparison of Section 5, no model, budget or seed was frozen in advance, and nothing here should be read as ranking agents or models in general.

125 Six of the seven reach micro F1 0.79–0.89, all far above every baseline above, with zero in-
 126 valid predictions across all 348 units. GPT-6-Astra is highest overall (0.892) and on macro
 127 F1 (0.701), narrowly ahead of Claude Code under Opus 5 (0.880) and Codex under GPT-5.6-
 128 Sol (0.875, highest on localization at 0.938); Claude Code under Sonnet 5 (0.856) and the two
 129 Antigravity/Gemini runs (0.797, 0.795) complete the six. All three agents reach per-label F1
 130 1.00 on basmala_benign and isti3adha_benign in at least one run; spelling_benign is
 131 the clearest shared weakness, F1 0.15–0.67 across all six, and the two single-example labels
 132 (insertion_mistake, omission_corrected) are where every run is weakest overall, unsurpris-
 133 ingly given one teaching example each. Gemini 3.1 Pro recovers isti3adha_benign perfectly but
 134 only 5 of 18 basmala_benign events, consistent with an opening-formula handler that returns one
 135 label per unit where a compound opening carries two.

136 The seventh run is an informative failure. gpt-oss-120B’s solution never normalizes Arabic orthogra-
 137 phy before diffing, so ordinary hamza and alif-form differences register as substitution_mistake
 138 on nearly every token (545 of 567 predicted events carry that one label); it also treats every opening-
 139 formula unit as clean, missing all 47 basmala_benign/isti3adha_benign events, and produces
 140 22 invalid events from a schema mismatch on its insertion/omission spans. The result, micro F1 0.143
 141 and a 68% clean-flag rate, is worse than either non-agent baseline: the task’s difficulty is not diffing,
 142 and a diff-shaped answer under the same instructions, guide and examples as the six successful runs
 143 can still fail on the comparison step that Section 2’s rubric assumes is already solved.

144 5 Limitations and what comes next

145 The annotated set is purposive and small, one learner contributes 46 of the 100 recordings, and the
 146 label distribution is heavily skewed, so macro F1 here is a coverage check rather than a stable estimate.
 147 More consequentially, all 100 selected transcripts already appear as inputs in a public release of the
 148 same recordings, and twelve appear in an earlier machine-labeled pilot. The adjudications and the
 149 selection membership are private, but the inputs are not, and runtime isolation cannot establish absence
 150 of prior exposure. An unseen-input claim requires a separate corpus. The prepared development
 151 release contains 127 cases and 888 units, including 44 of 63 newer recordings. It permits shared clean
 152 ayahs and different error variants, but excludes complete gold copies and copies of event-bearing gold
 153 chunks. The one local ASR repair appears non-Quranic and is excluded. Remaining new recordings
 154 are not a frozen hidden test.

155 We plan to compare agents under the same unlabelled training data, rubric, examples, tools and
 156 budget. Agents may annotate, train models or write rules; annotation is an available strategy, not
 157 the experimental contrast. The pilot above is a first, informal instance of this design, run to check
 158 that the harness and isolation work end to end before committing to the preregistered protocol: one
 159 run per model across seven agent/model pairs, an ad hoc 20-minute budget rather than a frozen
 160 one, no repeats, and no fixed choice yet of model roster, seed or annotation-delivery requirement.
 161 Isolation differed by runtime. Codex and Claude Code each ran non-interactively in a freshly created,
 162 git-free directory outside this repository, under their sandbox’s default network restriction, with an
 163 isolated credentials-only home directory carrying none of the operator’s prior session history, cache or
 164 goals, one fresh instance per model. Antigravity ran in a freshly opened window and freshly opened
 165 directory per model; its network access was not independently instrumented, and its selected model
 166 per run is taken on the operator’s report rather than verified from outside the application. All seven saw
 167 only the unlabelled release, the guide and the teaching examples, never gold, and all seven produced
 168 their frozen solution before any gold scoring occurred. A high score supports the combined process,
 169 not each training label: code may ignore or correct its labels. Generated annotations need independent
 170 review before being treated as reliable; a fully human-labelled training pool is unnecessary. Gold
 171 answers, selected test bundles and review history stay outside development environments, with shared
 172 clean inputs and prior public exposure disclosed. The repeated-run comparison, with budgets, models
 173 and the annotation-delivery requirement fixed in advance, remains future work.